Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Recursión: caso base, caso recursivo, la pila de llamadas y cómo estimar el costo con el árbol de recursión
- Generar todas las opciones: subconjuntos, permutaciones y combinaciones
- Backtracking: construir una solución paso a paso y retroceder apenas se sabe que no se puede completar

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Todas las contraseñas

Listar todas las contraseñas de longitud 3 con las letras *a* y *b* se hace con tres ciclos anidados, uno por posición: *aaa*, *aab*, . . . , *bbb*.

¿Y si la longitud n es un dato de entrada?

No se puede escribir un número variable de ciclos. . .

. . . pero una función puede decidir la posición i y dejar las posiciones $i+1, \ldots, n$ a una llamada de sí misma. Hoy: la recursión, cómo recorrer con ella todas las opciones y cómo evitar las que ya sabemos que no sirven.

Recursión

Una función que se llama a sí misma

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Tres elementos

Predicado: qué calcula la función, dicho en palabras, y de qué argumentos depende.
Casos base: argumentos que se resuelven sin llamarse a sí misma.
Paso recursivo: cómo se arma la respuesta con llamadas a argumentos más chicos, que tienen que terminar en un caso base.

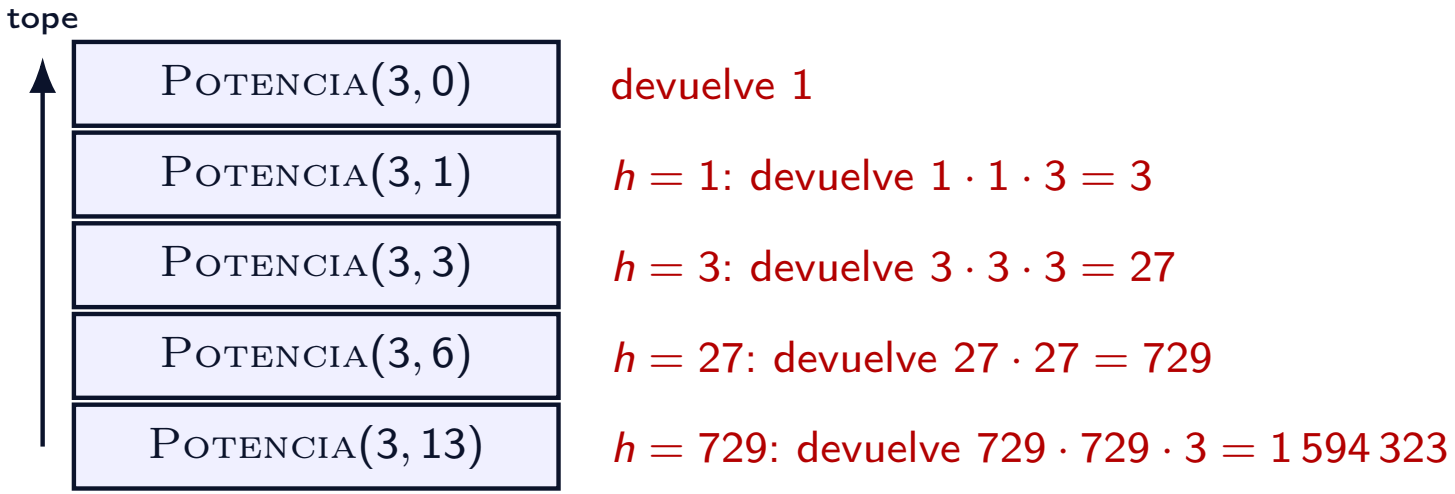
Algoritmo 3: $\textsc{Potencia}(a, n)$ — predicado: "el valor de a^n", con $n \geq 0$

---

si $n = 0$ **entonces devolver** 1
$h \leftarrow \textsc{Potencia}(a, \lfloor n/2 \rfloor)$
si *n es par* **entonces devolver** $h \cdot h$ **en otro caso devolver** $h \cdot h \cdot a$

$a^{13} = (a^6)^2 \cdot a$, $a^6 = (a^3)^2$, $a^3 = (a^1)^2 \cdot a$: cada llamada divide n entre 2.

La pila de llamadas



Cada llamada tiene su propio **marco**: su copia de a, n y h, y el punto donde tiene que continuar. Solo la de arriba está activa; las de abajo esperan su resultado.

Es la pila de la semana 4

La última llamada en entrar es la primera en terminar: al retornar, su marco se desapila y la llamada de abajo continúa con h ya calculado.

¿Qué pasa si nunca se llega al caso base?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dos formas de fallar

Falta el caso base, o las llamadas no se acercan a él: con $n < 0$, $\lfloor n/2 \rfloor$ se queda en -1 para siempre.

¿Qué hace el programa en ese caso?

Apila marcos hasta agotar la memoria de la pila: *stack overflow*

La pila de llamadas tiene un tamaño fijo, de unos pocos megabytes. Incluso una recursión correcta puede agotarla si es muy profunda: una función que baja de n a $n - 1$ con $n = 10^6$ apila un millón de marcos.

¿Cuánto cuesta una función recursiva?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Se suma el trabajo de todas las llamadas

El **árbol de recursión** tiene un nodo por llamada. El costo total es la suma del trabajo que hace cada llamada fuera de sus llamadas recursivas.

Caso recursivo	**Llamadas**	**Costo**
$a \cdot \textsc{Potencia}(a, n-1)$	$n+1$, en cadena	$O(n)$
$h \leftarrow \textsc{Potencia}(a, \lfloor n/2 \rfloor)$; $h \cdot h$	$\approx \lg n$, en cadena	$O(\lg n)$
$\textsc{Potencia}(a, \lfloor n/2 \rfloor) \cdot \textsc{Potencia}(a, \lfloor n/2 \rfloor)$	dos por nivel	?

La última vuelve a ser $O(n)$

Dos llamadas por nivel durante $\lg n$ niveles dan $2^{\lg n} = n$ hojas.

Generar todas las opciones

Subconjuntos: una decisión por elemento

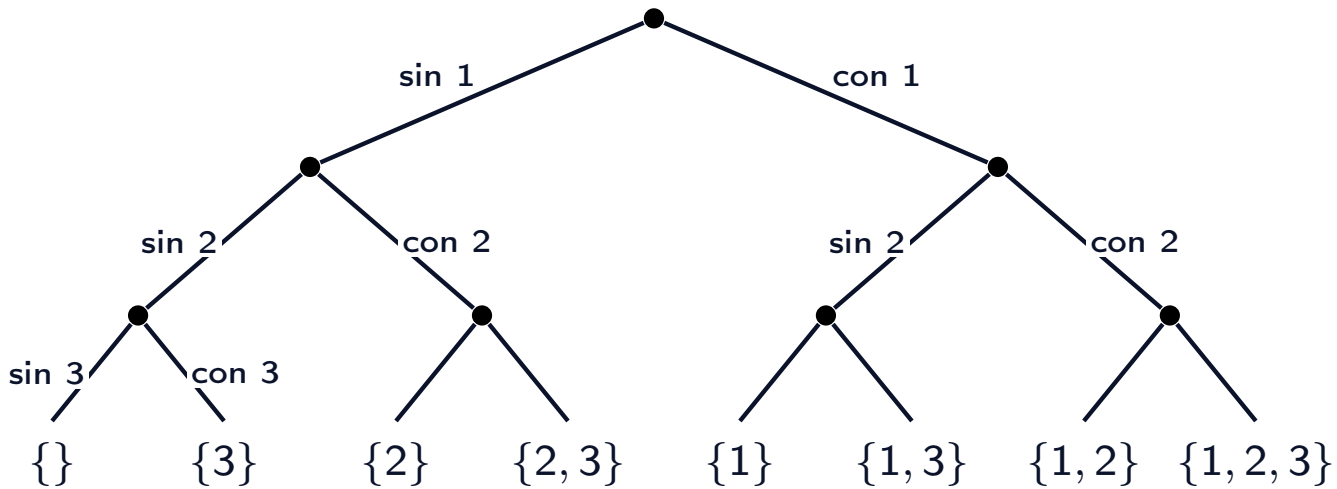
Algoritmo 5: SUBCONJUNTOS(i) — decide los elementos $i, \ldots, n$

si $i = n + 1$ **entonces**
    imprimir los k con $elegido[k] =$ VERDADERO
    devolver

$elegido[i] \leftarrow$ FALSO, SUBCONJUNTOS($i + 1$)
$elegido[i] \leftarrow$ VERDADERO, SUBCONJUNTOS($i + 1$)

Se llama con SUBCONJUNTOS(1). Cada llamada hace el trabajo de uno de los ciclos anidados: prueba las dos opciones del elemento i y deja el resto a la recursión. Es exactamente el problema de las contraseñas, con {FALSO, VERDADERO} en lugar de $\{a, b\}$.

El árbol de recursión de Subconjuntos



Una hoja por subconjunto: 2^n hojas y $2^{n+1} - 1$ llamadas en total. Imprimir cada uno cuesta $O(n)$, así que el total es $O(n\,2^n)$: con $n = 20$ ya son más de un millón de subconjuntos.

Permutaciones

Algoritmo 6: PERMUTACIONES(p) — p: la parte ya armada

```
si |p| = n entonces
    imprimir p
    devolver
para v ← 1 a n hacer
    si no usado[v] entonces
        usado[v] ← VERDADERO,  agregar v al final de p        // hacer
        PERMUTACIONES(p)                                      // explorar
        quitar el último de p,  usado[v] ← FALSO              // deshacer
```

Permutaciones

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Por qué hay que deshacer?

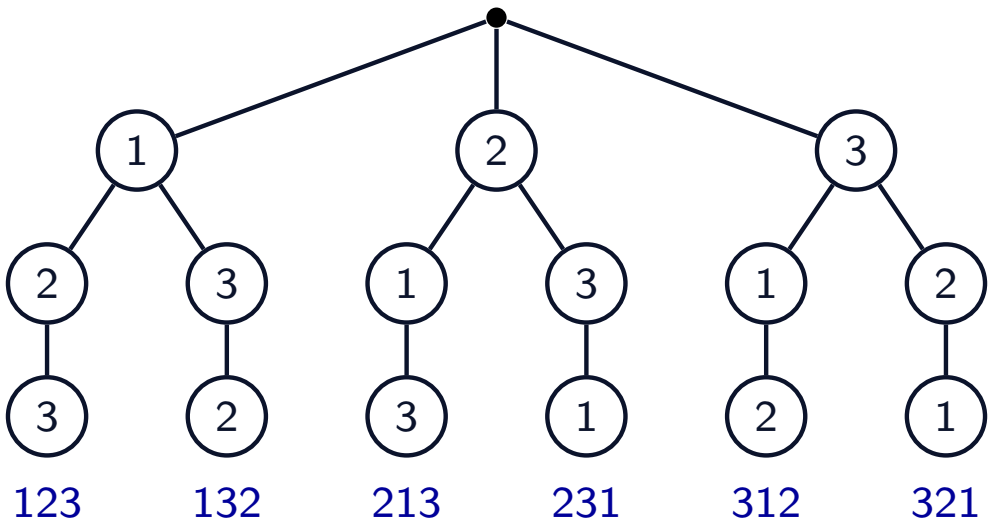
Permutaciones

¿Por qué hay que deshacer?

Para que la siguiente opción parta del mismo estado

Al volver de la recursión, p y *usado* quedan como estaban antes de probar v, y el ciclo puede probar $v + 1$. Este patrón, hacer–explorar–deshacer, es el corazón de todo lo que sigue.

El árbol de las permutaciones



n opciones en el primer nivel, $n-1$ en el segundo...: $n!$ hojas. Como $10! \approx 3{,}6 \cdot 10^6$, listar todas las permutaciones solo es posible para n de hasta 10 u 11.

Combinaciones: elegir k de n

Algoritmo 8: COMBINACIONES$(c, desde)$

si $|c| = k$ **entonces**
  imprimir c
  devolver

para $v \leftarrow desde$ **a** n **hacer**
  agregar v a c, COMBINACIONES$(c, v+1)$, quitar v de c

Una primera poda

Si quedan menos números que los que faltan ($n - v + 1 < k - |c|$), ese camino no termina en ninguna combinación: se corta el ciclo. Con $n = 20$ y $k = 10$, las llamadas bajan de 616 666 a 352 716.

Backtracking

Construir, verificar, retroceder

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Fuerza bruta, apoyada en la recursión

Se construye la solución una decisión a la vez. Apenas una solución parcial ya no puede convertirse en solución, se descarta sin explorar nada de lo que venía debajo: se deshace la última decisión y se prueba la siguiente opción. Se **retrocede**.

Construir, verificar, retroceder

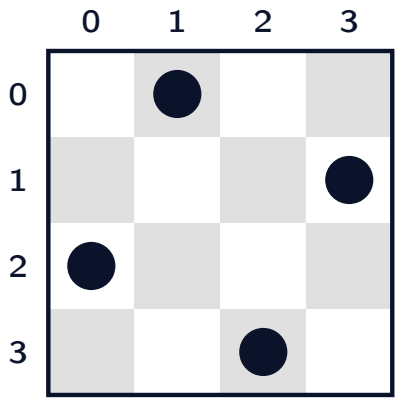
Algoritmo 10: BACKTRACKING(*parcial*)

si *parcial está completa* **entonces**
    procesar la solución
    devolver

para cada *opción o para la siguiente decisión* **hacer**
    si *parcial con o todavía puede completarse* **entonces**
        aplicar *o*, BACKTRACKING(*parcial*), deshacer *o*

Es el esquema de las permutaciones, con una verificación. Cada verificación que falla es una **poda**: todo un subárbol del árbol de recursión que no se visita.

N reinas



El problema

Colocar n reinas en un tablero de $n \times n$ sin que dos se ataquen: ninguna pareja en la misma fila, columna o diagonal.

El modelo

Hay exactamente una reina por fila, así que se decide, fila por fila, en qué columna va. Las reinas (f_1, c_1) y (f_2, c_2) comparten diagonal si $f_1 - c_1 = f_2 - c_2$, o antidiagonal si $f_1 + c_1 = f_2 + c_2$.

Con tres arreglos de booleanos (columnas, diagonales $f - c$ y antidiagonales $f + c$ ocupadas), cada verificación cuesta $O(1)$.

N reinas: pseudocódigo

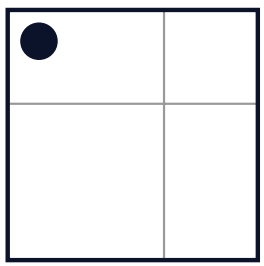
Algoritmo 12: REINAS(f) — coloca las reinas de las filas $f, \ldots, n-1$

si $f = n$ **entonces**
    registrar la solución *pos*
    devolver

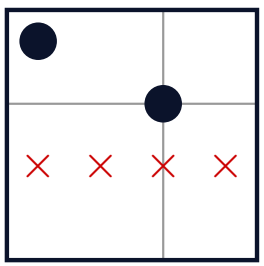
para $c \leftarrow 0$ **a** $n-1$ **hacer**
    si ***no*** $col[c]$ ***y no*** $d_1[f-c+n]$ ***y no*** $d_2[f+c]$ **entonces**
        $col[c],\ d_1[f-c+n],\ d_2[f+c] \leftarrow$ VERDADERO, $pos[f] \leftarrow c$
        REINAS($f+1$)
        $col[c],\ d_1[f-c+n],\ d_2[f+c] \leftarrow$ FALSO

$f - c$ va de $-(n-1)$ a $n-1$: se le suma n para usarlo como índice. Se llama con REINAS(0) y todos los arreglos en FALSO.

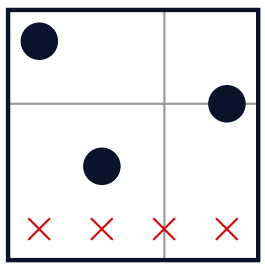
N reinas con $n = 4$



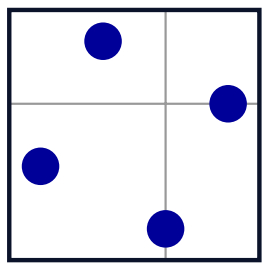
$(0, 0)$



$(0, 0), (1, 2)$: la fila 2 no tiene lugar



$(1, 3), (2, 1)$: la fila 3 no tiene lugar



se retrocede hasta la fila 0: solución $(1, 3, 0, 2)$

Con la reina de la fila 1 en la columna 2, las cuatro casillas de la fila 2 están atacadas: se deshace esa decisión.

Con la reina de la fila 0 en la columna 0 no hay solución: se prueba la columna 1, y ahí sí. Para encontrar las dos soluciones, $(1, 3, 0, 2)$ y su reflejo $(2, 0, 3, 1)$, REINAS se llama solo 17 veces.

Contar en lugar de listar

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Predicado: $\textsc{Contar}(f)$ es la cantidad de formas de completar el tablero si las filas $0, \ldots, f-1$ ya tienen sus reinas

Algoritmo 14: $\textsc{Contar}(f)$

```
si f = n entonces devolver 1   (un tablero completo es una forma)
total ← 0
para c ← 0 a n − 1 hacer
    si la casilla (f, c) no está atacada entonces
        colocar una reina en (f, c)
        total ← total + Contar(f + 1)
        quitar la reina de (f, c)
devolver total
```

¿Cuánto ahorra la poda?

Para $n = 8$	**Casos**
Una reina por fila, en cualquier columna: 8^8	16 777 216
Además en columnas distintas: 8!	40 320
Llamadas de REINAS (backtracking)	2 057
Soluciones	92

¿Entonces backtracking es polinomial?

No: la poda no cambia el peor caso

Solo evita ramas que ya se sabe que fallan, y cuánto ahorra depende del problema. En el peor caso sigue siendo exponencial: por eso estos problemas vienen con n chico, del orden de 15 o 20.

Suma de subconjuntos, con poda

¿Qué subconjuntos de $a = \{3, 5, 6, 7\}$ suman exactamente $T = 15$?

Algoritmo 16: $\textsc{Suma}(i, s)$ — $a_1 \leq a_2 \leq \cdots \leq a_n$, todos positivos

si $s = T$ **entonces**
    registrar los elegidos
    devolver

si $i > n$ ***o*** $s + a_i > T$ **entonces devolver**
elegir a_i, $\textsc{Suma}(i + 1, s + a_i)$, deshacer
$\textsc{Suma}(i + 1, s)$

Ordenar primero hace más fuerte la poda

Si a_i ya no cabe en lo que falta, ninguno de los siguientes cabe: se descarta el subárbol completo. Aquí la única respuesta es $\{3, 5, 7\}$.

Cierre de la sesión

- Una función recursiva tiene un predicado (qué calcula, en palabras), casos base y un paso recursivo que llega a ellos. Cada llamada ocupa un marco en la pila de llamadas; demasiada profundidad produce *stack overflow*.
- El costo es la suma sobre el árbol de recursión: su forma (una o dos llamadas por nivel) lo decide todo.
- Subconjuntos (2^n), permutaciones ($n!$) y combinaciones ($\binom{n}{k}$) se generan con el patrón hacer–explorar–deshacer.
- Backtracking: el mismo patrón, verificando cada decisión y podando las ramas que no se pueden completar (N reinas, suma de subconjuntos). El peor caso sigue siendo exponencial.

Logros de la sesión

- Sabemos escribir funciones recursivas a partir de su predicado, casos base y paso recursivo, y seguir su ejecución en la pila de llamadas.
- Sabemos estimar el costo de una recursión con su árbol de recursión.
- Sabemos generar subconjuntos, permutaciones y combinaciones, y resolver problemas con backtracking y poda.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias